Rockchip RKStudio 第三方算法开发指南

文件标识：RK-KF-SF-044

发布版本：V2.0.0

日期：2025-03-11

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 **© 2025 **瑞芯微电子股份有限公司**

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-0591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档旨在详细说明 RKStudio_Tool 中导入第三方算法的具体方法，帮助开发者实现算法的接入、调试、部署等全流程操作，为算法集成提供清晰的指导与规范。

产品版本

芯片名称
RK2118

读者对象

本文档（本指南）主要适用于以下工程师：

音频工程师

技术支持工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	廖华平	2024-05-23	初始版本
V2.0.0	廖华平	2025-03-11	修改接口和调用方式

目录

- Rockchip RKStudio 第三方算法开发指南
  - RKStudio Tool端界面配置
    - 文件说明
    - 框架差异
    - JSON文件通用配置
    - V1 Param介绍
    - V2 Param介绍
    - 多算法配置与生效规则
    - 配置注意事项
  - 算法接口
    - BlockPro
    - BlockAlgo
    - 第三方算法接口注册
  - 通用接口
    - WAV分区文件管理
  - 附录：注册代码实例

RKStudio Tool端界面配置

文件说明

在工具的CustomAlgorithms目录下，存在两种版本的算法参数配置文件：

- custom_algo_v1.json：基于 v1 框架（适用于 V2.0.0 文档版本之前的框架）编写。
- custom_algo_v2.json：基于 v2 框架编写。

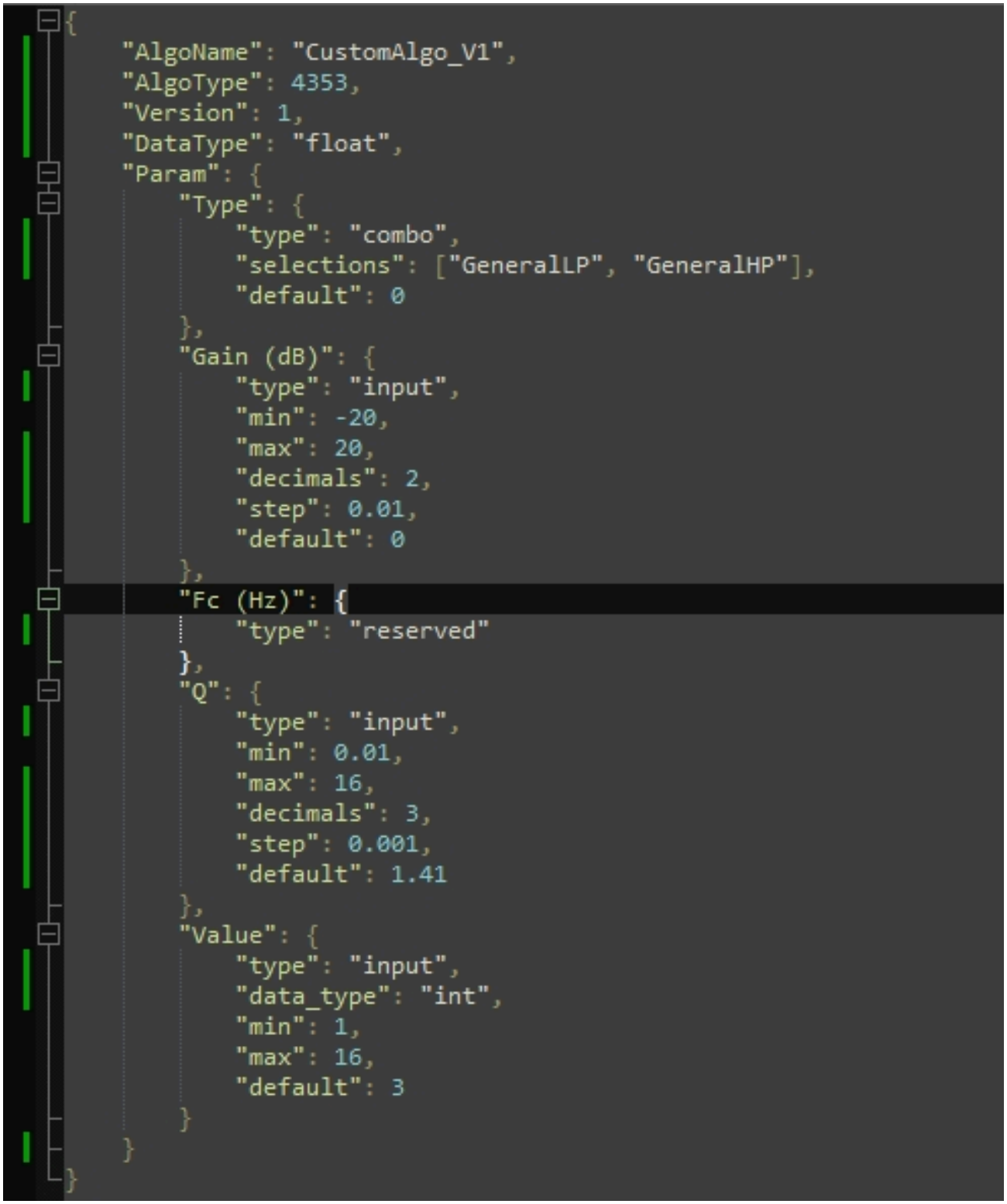
框架差异

- **v1 框架**
  - 部分属性名已发生变化，但兼容旧版本。
  - JSON配置较为简单，配置界面呈现为控件列表，**适合简单的参数配置场景**。
- **v2 框架**
  - JSON配置相对复杂，配置界面采用多 Tab 页与多控件的布局形式，**适用于复杂的参数配置场景**。

开发者可根据实际需求，选择匹配的框架版本进行配置。

JSON文件通用配置

- `AlgoName`： 算法名称，作为控件栏中显示的标识，必须保证全局唯一性，不可包含空格及其他异常字符，
- `AlgoType`： 算法类型标识，用于与板端算法函数注册时的 `algoType` 进行匹配，必须保证全局唯一性，自定义算法类型取值范围严格限制在 `0x1100~0x11FF` 。算法注册的详细说明参见文档 2.3 章节。
- `Version`： JSON框架版本，配置1或者2。如未配置，默认版本为1。
- `DataType`： 参数数据类型，配置为int/int32/int32_t/uint/uint32/uint32_t/float。如未配置，默认数据类型为float。
- `Param`： V1和V2版本的JSON框架都有 `Param` 属性，包含算法相关的参数配置。每一个属性对应一个参数位，支持int/int32/int32_t/uint/uint32/uint32_t/float等数据类型，每个参数的数据宽度为4byte。最终生成的参数按照JSON中定义的参数顺序，依次放到算法的参数中。也就是在算法调用注册的函数时，就能在 `algoSt->pParam` 指针中获取到算法的参数， `algoSt->nParamSize` 对应的参数内存的大小，单位为Byte。



```json
{
    "AlgoName": "CustomAlgo_V1",
    "AlgoType": 4353,
    "Version": 1,
    "DataType": "float",
    "Param": {
        "Type": {
            "type": "combo",
            "selections": ["GeneralLP", "GeneralHP"],
            "default": 0
        },
        "Gain (dB)": {
            "type": "input",
            "min": -20,
            "max": 20,
            "decimals": 2,
            "step": 0.01,
            "default": 0
        },
        "Fc (Hz)": {
            "type": "reserved"
        },
        "Q": {
            "type": "input",
            "min": 0.01,
            "max": 16,
            "decimals": 3,
            "step": 0.001,
            "default": 1.41
        },
        "Value": {
            "type": "input",
            "data_type": "int",
            "min": 1,
            "max": 16,
            "default": 3
        }
    }
}
```

图 1-1 V1 JSON文件

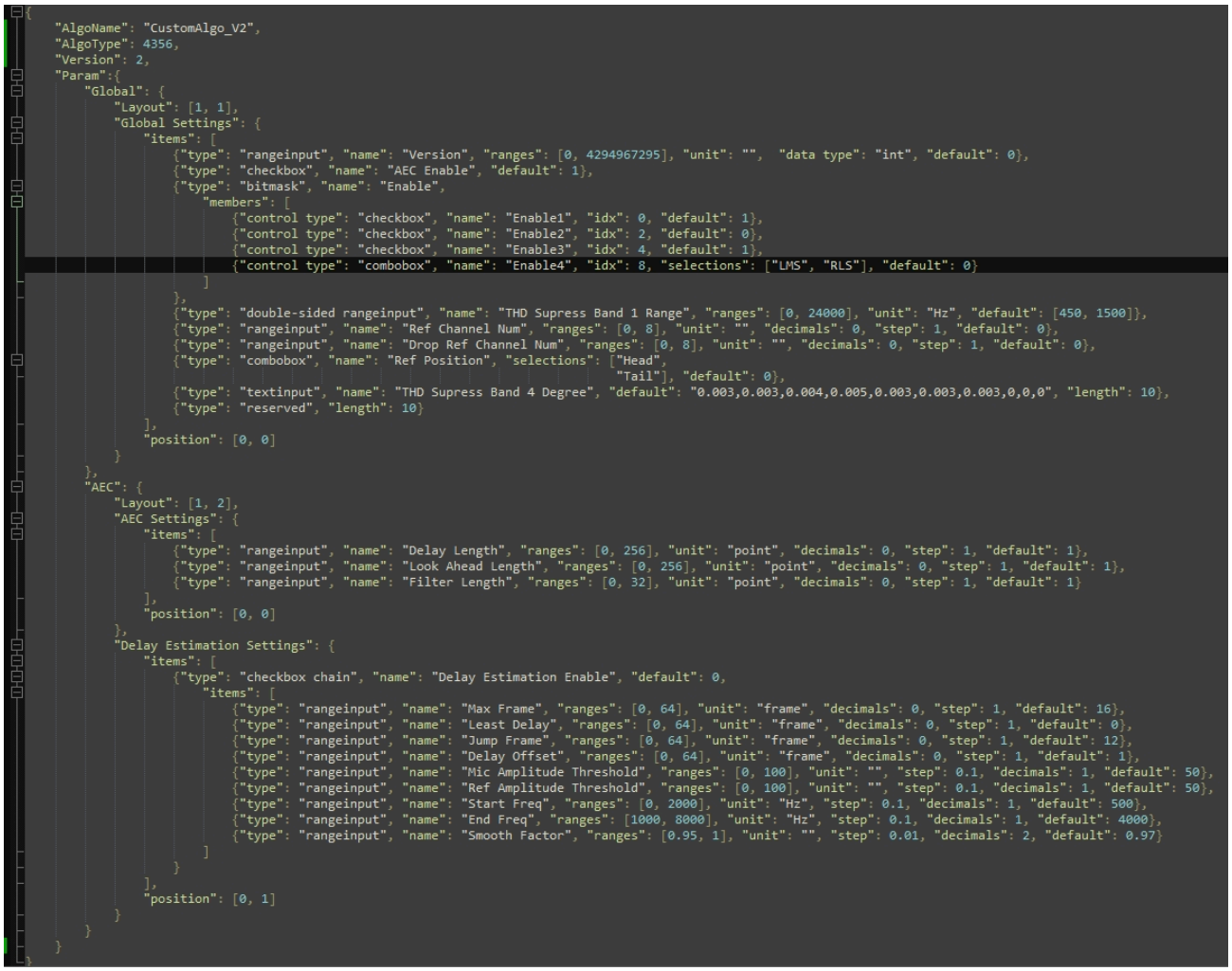


图 1-2 V2 JSON文件

V1 Param介绍

以custom_algo_v1.json为例，配置的属性名对应配置菜单中的参数名称，如第一个名称 `Type` ， 对应的就是UI配置表中的Type名称。每个参数对应的属性如下：

- `type` ： 输入的UI交互类型：
  - `combobox` ： 选择框形式，支持预设多个值，列表存于 `"selections": ["GeneralLP", "GeneralHP"]` 中， `default` 为默认值对应的索引，传递的参数值是对应的索引值，如选择 `GeneralLP` ，那么传递的参数值就是0。
  - `rangecombo` ： 根据设置的 `min` 和 `max` 值，自动生成 `selections` 列表，列表中的值会在 `min` 和 `max` 间均匀分布， `default` 为默认值对应的索引，传递的参数值是对应的索引值。
  - `rangeinput` ： 输入框形式，支持输入 `min` 和 `max` 范围内的数值， `default` 为默认值。
  - `reserved` ： 只做参数占位，不做任何功能，不在配置框中显示。方便参数空间不连续，或者不需要用户配置时使用，默认值为0。
- `data_type`：数据类型，若未配置，则使用1.3中配置的 `DataType` 。如 `DataType` 未配置，则默认使用float。支持的数据类型为int/int32/int32_t/uint/uint32/uint32_t/float。

- `min`： 最小值，input类型为 `rangeinput` 和 `rangecombo` 时需要配置。
- `max`： 最大值，input类型为 `rangeinput` 和 `rangecombo` 时需要配置。
- `decimals`：小数位数，input类型为 `rangeinput` 时有效。如未配置，则默认为2。
- `step`： 上下键切换数值时，每次变化的步长，input类型为 `rangeinput` 时有效。如未设置，则默认为0.01。
- `selections`： 选择框的选项列表，input类型为 `combobox` 时需要配置。

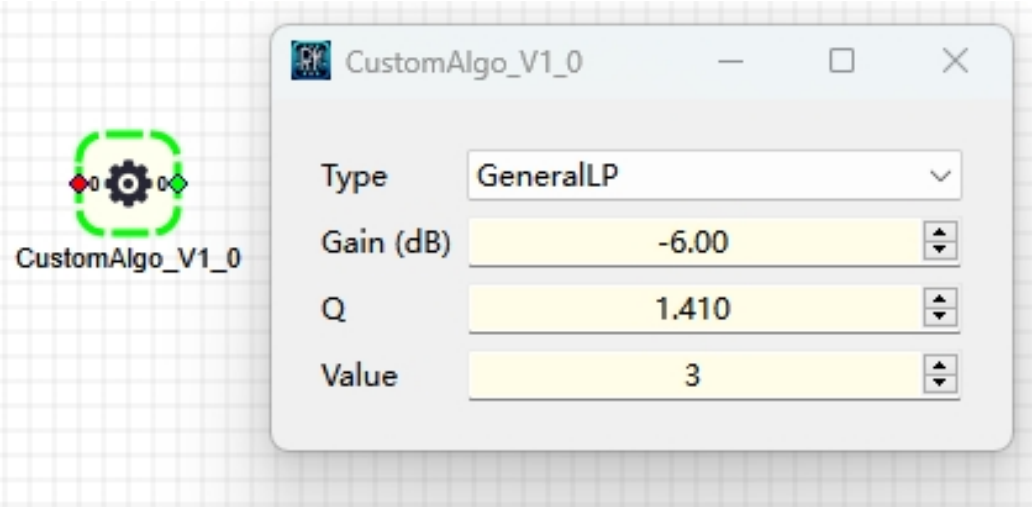


图 1-3 custom_algo_v1.json对应的配置界面

V2 Param介绍

以custom_algo_v2.json为例， `Global` 和 `AEC` 对应两个 Tab 页，每个 Tab 页包含多个控件，每个控件分别映射一个参数。其中， `AEC` Tab 页中各参数的属性如下：

- `Layout`：[1, 2],表示当前AEC页面下，布局为1行2列。
- `AEC Settings` 和 `Delay Estimation Settings` 表示两个不同的布局名称。
  - `position`：[0, 0]表示该布局的位置为第0行第0列。如未配置，默认位置为[0, 0]。
  - `items` 定义控件参数。

以 `Delay Estimation Settings` 为例，每个参数对应的属性如下：

- `type`： 输入的UI交互类型
- `checkbox chain`： 复选框链，如果选中，则显示全部 `items` ，未选中则全部隐藏。选中则为1，否则为0。
- `combobox`： 选择框形式，可以选择预设的多个值，列表存于"selections": ["LMS", "RLS"] 中。 `default` 为默认值对应的索引，传递的参数值是对应的索引值，如选择了"LMS"，那么传递的参数值就是0。
  - `rangecombo`： 会根据设置的 `min` 和 `max` 值，自动生成一个 `selections` 列表，列表中的值会在 `min` 和 `max` 之间均匀分布， `default` 为默认值对应的索引，传递的参数值是对应的索引值。

    - rangeinput： 输入框形式，可以输入 min 和 max 范围内的数值， default 为默认值。
    - double-sided rangeinput： 同时列出两个输入框，除了 default 外，共用其他配置。一般用于成对的范围值设置，输出为两个参数。其他的用法和 rangeinput 相同。
    - reserved： 只做参数占位，不做任何功能，也不在配置框中显示。方便参数空间不连续，或者不需要用户配置时使用，默认值为0，数据长度为 length 。
    - checkbox： 复选框形式，选中为1，否则为0。
    - textinput：从文本文件中导入对应的数据，数据长度为 length 。
    - bitmask： **定义可以配置32个bit位的参数类型，数据类型固定为uint32，不支持自行配置数据类型。**
    - members： 定义选项列表，相关属性如下：
        - control type：可配置为 checkbox 和 combobox ， 分别对应复选框和选择框，选择框只能有两个配置，分别对应0和1。
        - name：选项名称。
        - default：默认值，0或1。
        - idx： 对应的bit位索引，从0到31。如若选中，该bit位的值为1，否则为0。
        - selections： 选择框的选项列表， control type 类型为 combobox 时需要配置，只支持两个配置项。
- data_type：传输的数据类型，若未配置，则使用1.3中配置的 DataType ， 如 DataType 未配置，则默认使用float。支持的数据类型为int/int32/int32_t/uint/uint32/uint32_t/float。
- min： 最小值，input类型为 rangeinput 和 rangecombo 时需要配置。
- max： 最大值，input类型为 rangeinput 和 rangecombo 时需要配置。
- decimals：小数位数，input类型为 rangeinput 时有效。若未配置，则默认为2。
- step： 上下键切换数值时，每次变化的步长，input类型为 rangeinput 时有效，若未设置，则默认为0.01。
- selections： 选择框的选项列表，input类型为 combobox 时需要配置。

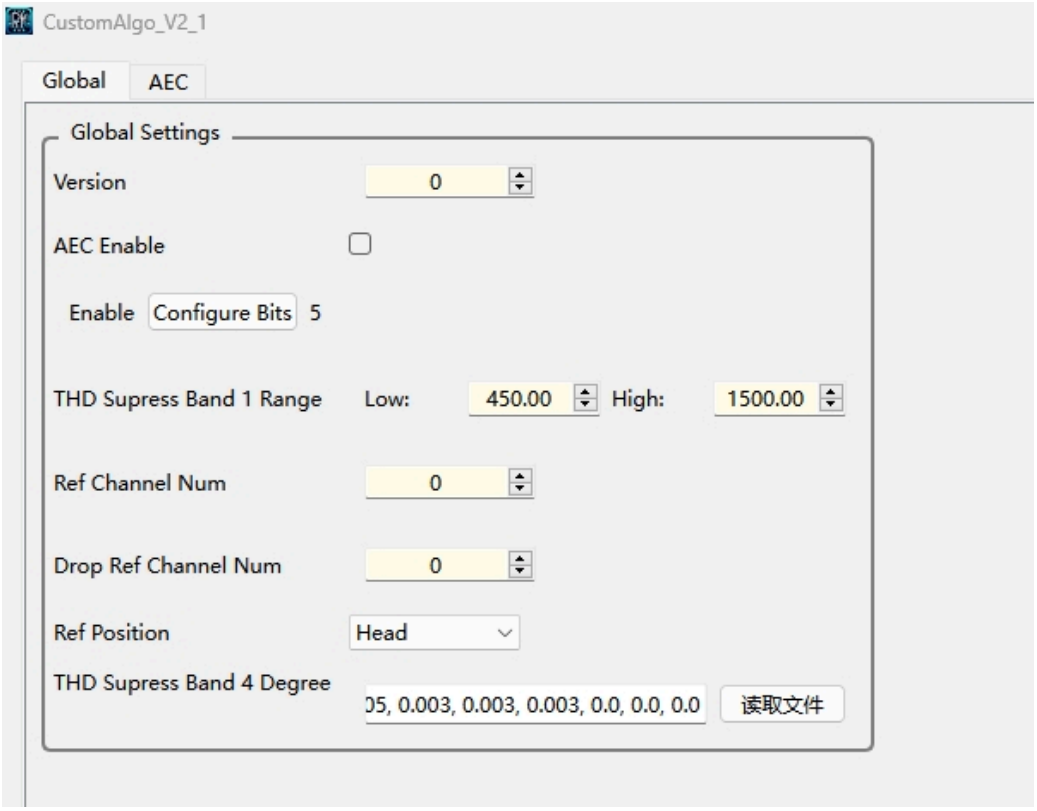


图 1-4 custom_algo_v2.json对应的配置界面

多算法配置与生效规则

- 若存在多个算法，可添加多个 JSON 文件，重新打开工程图纸后即可生效。
- 新添加算法控件/修改算法名和算法类型需重新打开工具才能生效。
- 参数内容支持动态修改，重新拖出控件即可生效，无需重新打开工具。

配置注意事项

- 同一个算法控件可以拖出多个，但控件的名字不能自定义，且控件名字的主体需与属性 `AlgoName` 保持一致。
- 定义的 JSON 文件必须严格遵循 JSON 格式要求，否则工具会报错。可通过https://www.json.cn/验证文件格式的正确性。
- 各个属性的名字必须严格按照文档中的描述填写，否则算法控件会添加失败。

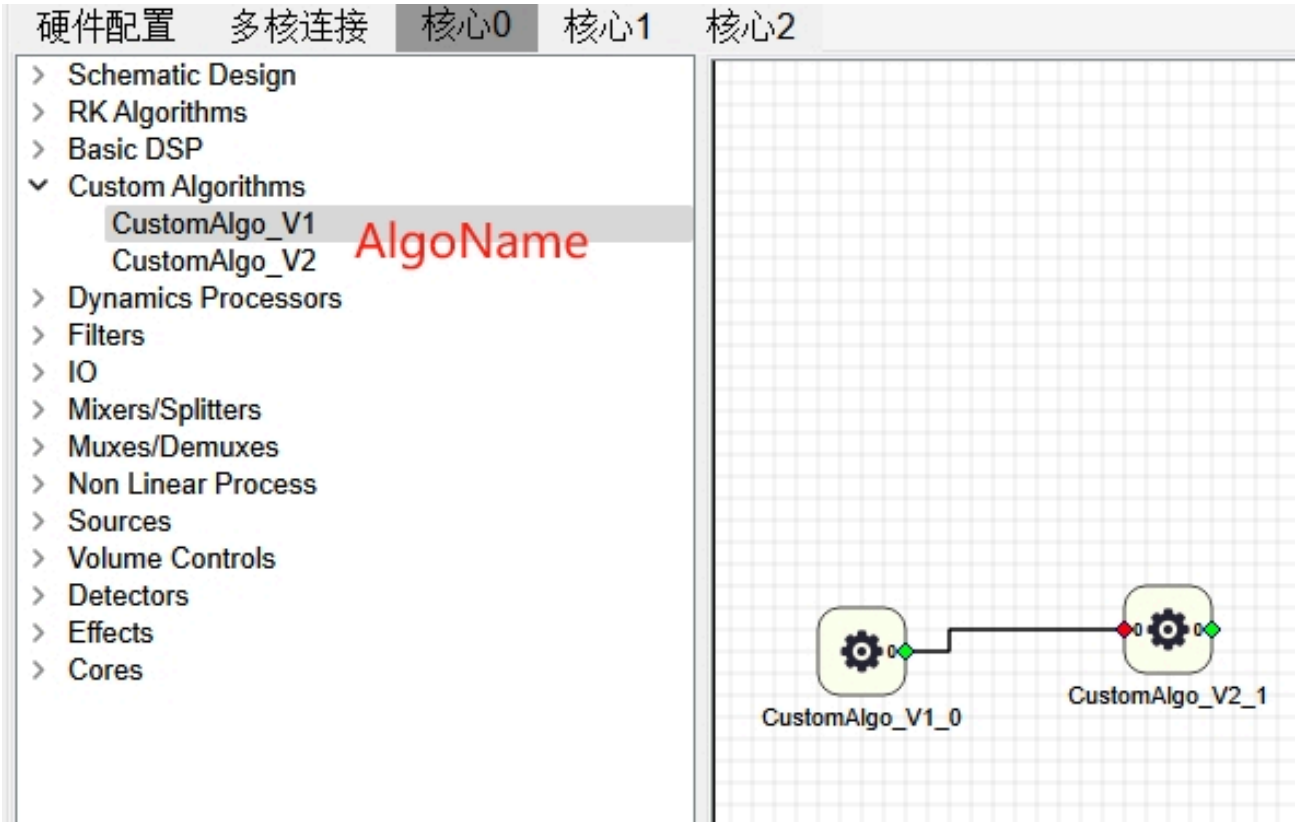


图 1-5 控件在工具栏上的位置

算法接口

目前支持自定义算法接口，需要开发者封装接口，自行编译并注册到RKStudio算法中。**本文档V1.0.0介绍的结构体、函数名、参数类型和相关用法均有变化，请参考V2.0.0版本说明进行修改，避免编译和使用问题。**

BlockPro

`BlockPro` 结构体为各算法共用结构体，会存在 `BlockAlgo->pBlockPro` 中，常用的成员有采样率和帧长。相关函数是初始化 `BlockPro` 和销毁，一般初始化图纸会调用一次初始化函数，将 `BlockPro` 传入到各个算法模块中。接口调用已经集成在RKStudio的工程中，无需自行实现和调用，此处仅展示用法。

BlockPro结构体

```
typedef struct _BlockPro
{
    int             nSamplingRate;  /* sample rate of the audio block */
    int             bitRate;        /* bit rate of the audio block, no used now */
    int             nBlockSize;     /* block size of the audio block */
    float          *tmpBuffer;     /* 2 * nBlockSize * sizeof(float) bytes temporary buffer */
    FadeWinSt      *fadeWin;       /* fade in and out window */
    unsigned int    toolVersion;

    int             nReserve0;      /* reserved */
    int             nReserve1;      /* reserved */
} BlockPro;
```

- nSamplingRate： 当前算法的采样率，单位为Hz，这里使用的是RkStudioTool配置栏上配置的采样率。
- bitRate： 当前算法的bit位，这里使用的是RkStudioTool配置栏上配置的位宽。
- nBlockSize： 当前算法的帧长，这里使用的是RkStudioTool配置栏上配置的帧长。
- *tmpBuffer： 指向一个临时缓冲区，所有算法都可以临时调用。大小为2 * nBlockSize * sizeof(float)。
- *fadeWin： 淡入淡出的窗，一般用于算法的淡入淡出处理。
- toolVersion： RkStudioTool的版本号，用于版本兼容。
- nReserve0： 预留成员，暂未使用。
- nReserve1： 预留成员，暂未使用。

BlockPro相关初始化和销毁函数

```
BlockPro *RKST_ALGO_InitBlock(int nSamplingRate,
                              int bitRate,
                              int nBlockSize,
                              FadeWinSt *fadeWin,
                              uint32_t toolVersion);
void RKST_ALGO_DestroyBlock(BlockPro *st);
```

BlockAlgo

BlockAlgo 作为算法调用的结构体，为各算法接口所共用。

BlockAlgo结构体

```
typedef struct _BlockAlgo
{
    int         nInputs;          /* number of input pins */
    int         nOutputs;         /* number of output pins */
    BlockPro   *pBlockPro;        /* pointer to the array of pBlockPro common structure */
    int         methodType;

    float      *pParam;           /* pointer to the parameter memory */
    int         nParamSize;       /* size of the parameter memory, unit is Byte */
    void       *pAlgoCore;        /* pointer to the algorithm memory */
} BlockAlgo;
```

- nInputs： 算法输入通道数。
- nOutputs： 算法输出通道数。
- *pBlockPro： BlockPro结构体指针，用于全局参数传递，结构体成员可查看2.1章节。
- *methodType： 当前算法类型，定义在rkstudio_process.h文件的RK_TYPE中。
- *pParam： 当前算法使用的参数，由框架自动分配和管理，调用算法接口时，已分配相关指针和内容。
- nParamSize： 参数内存大小，单位为Byte。
- *pAlgoCore： **算法核心内存指针，如开发者完成对GetAlgoSize函数的定义，框架自动分配对应空间。如未定义，则需要自行分配和释放空间。**

调用算法相关函数时，需注意：除 pAlgoCore 外， BlockAlgo *algoSt 结构体的所有成员均已由外部完成初始化，调用时无需重复执行初始化操作。

```
int RKST_ALGO_GetAlgoSize(BlockAlgo *algoSt);
int RKST_ALGO_Init(BlockAlgo *algoSt);
int RKST_ALGO_Destroy(BlockAlgo *algoSt);
int RKST_ALGO_SetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t nParamSize);
int RKST_ALGO_GetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t *nParamSize);
int RKST_ALGO_Process(BlockAlgo *algoSt,
                      float **dataIn,
                      float **dataOut);
```

- `RKST_ALGO_GetAlgoSize`： 获取pAlgoCore所需的空间大小，单位为Byte。若无需外部分配空间，返回值为 0。未注册此函数时，框架默认不分配空间。*pAlgoCore可由开发者自行操作。
- `RKST_ALGO_Init`： 用于执行算法初始化，调用时，BlockAlgo结构体所有成员已完成定义，*pParam和*pAlgoCore已分配空间(若GetAlgoSize返回0，则 `pAlgoCore` 需用户自行分配，且需在销毁时手动释放)。
- `BlockAlgo_Destroy`： 销毁init时释放的空间。
- `RKST_ALGO_SetParam`： 用于动态设置参数，nParamOff为参数在pParam中的偏移，每个偏移为4Byte，nParamSize为参数大小，单位为Byte。框架自动判断参数地址范围，自动修正 `nParamSize` 以确保不超过 `pParam` 大小。
- `RKST_ALGO_GetParam`： 用于动态获取参数，nParamOff为参数在pParam中的偏移，每个偏移为4Byte，nParamSize为参数大小，单位为Byte。框架自动判断参数地址范围，自动修正 `nParamSize` 以确保不超过 `pParam` 大小，并设置实际的nParamSize值。
- `RKST_ALGO_Process`： 算法执行函数，dataIn和dataOut是输入输出数据。数据结构式二维指针。例如 2 个通道时， `dataIn[0]` 指向第一通道数据， `dataIn[1]` 指向第二通道数据，所有通道数据独立存储（解交织形式）。为节省内存并避免数据拷贝， `dataOut` 可能复用 `dataIn` 的地址空间。若算法内部需多次调用 `dataIn` ，应确保 `dataOut` 相关数据赋值后，不再使用 `dataIn` 的原始数据。

第三方算法接口注册

调用 `RKST_ALGO_Register` 函数完成算法注册，**整个程序生命周期内仅允许注册一次**。algoType为算法类型，必须和工具自定义Json文件中的algoType保持一致，取值范围严格限制在 `0x1100~0x11FF` 之间，需保证值的唯一性，用于区分不同算法。函数相关定义和功能参见文档 2.3 章节。 `algoGetAlgoSize` 为可选函数，若未注册此函数，框架默认不分配 `*pAlgoCore` 空间。未注册时，必须将结构体对应成员设为 `NULL` ，否则会导致指针异常。

```
typedef struct
{
    int algoType;
    int (*algoInit)(BlockAlgo *algoSt);
    int (*algoSetParam)(BlockAlgo* algoSt,
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t nParamSize);
    int (*algoGetParam)(BlockAlgo* algoSt,
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t* nParamSize);
    int (*algoGetAlgoSize)(BlockAlgo *algoSt);
    int (*algoProcess)(BlockAlgo *algoSt,
                       float **dataIn,
                       float **dataOut);
    int (*algoDestroy)(BlockAlgo *algoSt);
} RkStudioFunc;

int RKST_ALGO_Register(RkStudioFunc* func);
```

通用接口

WAV分区文件管理

在工具-->文件分区配置中，支持配置多个文件，每个文件对应一个Idx。文件分区配置后，生成 `rkwav.img` 分区文件，将分区文件重新烧写到flash中。在算法接口中，通过调用公共接口获取文件地址和大小。

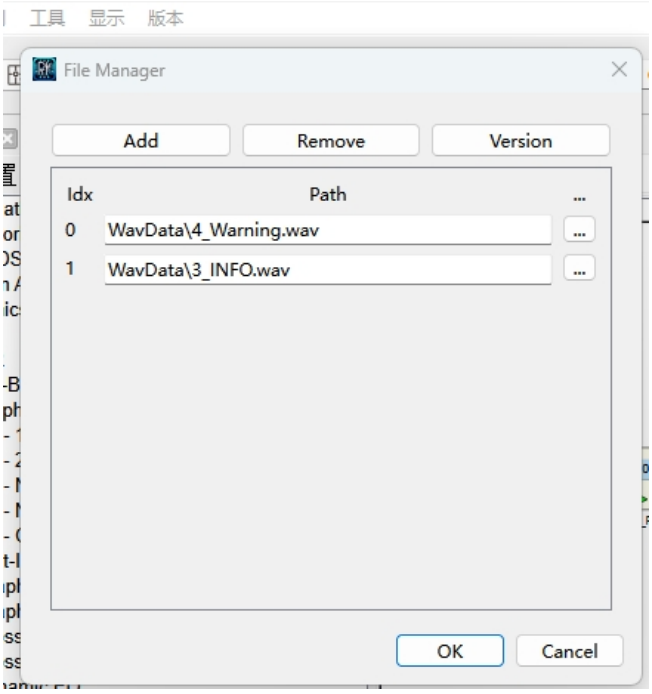


图 3-1 WAV分区文件管理界面

```
int RKST_WAV_GetAddr(uint32_t fileIdx, void **addr, uint32_t *size);
```

`fileIdx` 表示文件的idx， `addr` 表示文件地址， `size` 表示文件大小，返回0表示成功，其他值表示文件不存在。

示例代码如下：通过接口，获取idx为0的文件地址和大小，获取指针后以内存的形式访问文件内容。

```
void *addr;
uint32_t size;
int ret = RKST_WAV_GetAddr(0, &addr, &size);
```

附录：注册代码实例

相关结构体和接口定义如下，在工程
的 `rkstudio/basic/librkstudio_common.h` 和 `rkstudio/interface/rkstudio_process.h` 中。

```
typedef struct _BlockPro
{
    int            nSamplingRate;  /* sample rate of the audio block */
    int            bitRate;        /* bit rate of the audio block, no used now */
    int            nBlockSize;     /* block size of the audio block */
    float         *tmpBuffer;      /* 2 * nBlockSize * sizeof(float) bytes temporary buffer */
    FadeWinSt     *fadeWin;        /* fade in and out window */
    unsigned int   toolVersion;    /* RkStudioTool version */

    int            nReserve0;      /* reserved */
    int            nReserve1;      /* reserved */
} BlockPro;

#define RKST_ALIGN    8
#define RKST_ALIGN_UP(x) (((x) + (RKST_ALIGN -1)) & ~(RKST_ALIGN -1))
#if defined(_MSC_VER)
#define __RKST_ADDR_ALIGN__  __declspec(align(RKST_ALIGN))
#elif defined(__XTENSA__)
#define __RKST_ADDR_ALIGN__ __attribute__((aligned(RKST_ALIGN)))
#else
#define __RKST_ADDR_ALIGN__ ;
#endif

#pragma pack(push, RKST_ALIGN)
typedef struct _BlockAlgo
{
    int         nInputs;          /* number of input pins */
    int         nOutputs;         /* number of output pins */
    BlockPro   *pBlockPro;        /* pointer to the array of pBlockPro common structure */
    int         methodType;

    float      *pParam;           /* pointer to the parameter memory */
    int         nParamSize;       /* size of the parameter memory, unit is Byte */
    void       *pAlgoCore;        /* pointer to the algorithm memory */
} BlockAlgo;
#pragma pack(pop)


typedef struct
{
    int algoType;
    int (*algoInit)(BlockAlgo *algoSt);
    int (*algoSetParam)(BlockAlgo* algoSt,
```

```
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t nParamSize);
    int (*algoGetParam)(BlockAlgo* algoSt,
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t* nParamSize);
    int (*algoGetAlgoSize)(BlockAlgo *algoSt);
    int (*algoProcess)(BlockAlgo *algoSt,
                       float **dataIn,
                       float **dataOut);
    int (*algoDestroy)(BlockAlgo *algoSt);
} RkStudioFunc;

int RKST_ALGO_GetVersion(uint16_t *major,
                         uint16_t *minor,
                         const char **date,
                         const char **time);
BlockPro *RKST_ALGO_InitBlock(int nSamplingRate,
                              int bitRate,
                              int nBlockSize,
                              FadeWinSt *fadeWin,
                              uint32_t toolVersion);
void RKST_ALGO_DestroyBlock(BlockPro *blockProSt);

int RKST_ALGO_Register(RkStudioFunc* func);
int RKST_ALGO_GetAlgoSize(BlockAlgo *algoSt);
int RKST_ALGO_Init(BlockAlgo *algoSt);
int RKST_ALGO_Destroy(BlockAlgo *algoSt);
int RKST_ALGO_SetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t nParamSize);
int RKST_ALGO_GetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t *nParamSize);
int RKST_ALGO_Process(BlockAlgo *algoSt,
                      float **dataIn,
                      float **dataOut);
```

注册代码，可以在工程的rkstudio/librkstudio_custom.c中找到。

```c
typedef struct
{
    float gain;
    // todo
} CustomAlgoSt;

int Custom_Algo_SetParam(BlockAlgo* algoSt, void* param, uint32_t nParamOff, uint32_t nParamSize
{
    int chanNum = algoSt->nOutputs;
    float *pParam = (float *)algoSt->pParam;
    CustomAlgoSt *st = (CustomAlgoSt *)algoSt->pAlgoCore;

    if (param)
    {
        memcpy(pParam + nParamOff, param, nParamSize);
    }
    st->gain = pParam[1];
    // todo use the pParam
    return 0;
}

int Custom_Algo_GetAlgoSize(BlockAlgo *algoSt)
{
    return sizeof(CustomAlgoSt);
}

int Custom_Algo_Init(BlockAlgo *algoSt)
{

    CustomAlgoSt *st = (CustomAlgoSt *)algoSt->pAlgoCore;

    // algoSt->pParam ready now, don't need care
    return Custom_Algo_SetParam(algoSt, NULL, 0, 0);
}

int Custom_Algo_Process(BlockAlgo *algoSt, float **dataIn, float **dataOut)
{
    CustomAlgoSt *st = (CustomAlgoSt *)algoSt->pAlgoCore;
    int chanNum = algoSt->nInputs;
    int blockSize = algoSt->pBlockPro->nBlockSize;
    for (int chan = 0; chan < chanNum; chan++)
    {
        for (int i = 0; i < blockSize; i++)
```

```
            // just test for scale input data
            dataOut[chan][i] = dataIn[chan][i] * st->gain;
    }
    return 0;
}

int Custom_Algo_Destroy(BlockAlgo *algoSt)
{
    return 0;
}

int Custom_Algo_GetParam(BlockAlgo* algoSt, void* pParam, uint32_t nParamOff, uint32_t *nParamS
{
    memcpy(pParam, algoSt->pParam + nParamOff, *nParamSize);
    for (int i = nParamOff; i < *nParamSize / 4; i++)
        LOGD("param_%d:%f\n", i, algoSt->pParam[i]);
    return 0;
}

int librkstudio_register(void *param)
{
    int ret;
    RkStudioFunc func =
    {
        .algoType        = 0x1101,                      // must same with algoType in custom_a
        .algoInit        = Custom_Algo_Init,
        .algoSetParam    = Custom_Algo_SetParam,
        .algoGetParam    = Custom_Algo_GetParam,
        .algoGetAlgoSize = Custom_Algo_GetAlgoSize,     // You can also set this function as NU
        .algoProcess     = Custom_Algo_Process,
        .algoDestroy     = Custom_Algo_Destroy,
    };

    ret = RKST_ALGO_Register(&func);
    LOGD("register algoType = %d, ret = %d\n", func.algoType, ret);
    return ret;
}
```